Reflected XSS with AngularJS Sandbox Escape Without Strings

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting (XSS)
Technology	AngularJS
Attack Technique	Sandbox Escape
Injection Context	AngularJS Expression
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy

2. Description

This vulnerability occurs when an application reflects user-controlled input into an AngularJS expression context without appropriate protection.

AngularJS evaluates expressions within its own expression evaluation environment. Older versions of AngularJS implemented a sandbox intended to restrict access to potentially dangerous JavaScript functionality.

However, the AngularJS sandbox was not designed to be a security boundary. Attackers may exploit weaknesses in expression handling to escape restrictions and execute JavaScript.

This lab demonstrates an AngularJS sandbox escape technique that does not rely on conventional string literals.

3. Root Cause

The primary cause is the unsafe evaluation of untrusted user input as an AngularJS expression.

Contributing factors include:

- Direct insertion of user-controlled input into AngularJS expressions.
- Reliance on the AngularJS sandbox as a security control.
- Insufficient separation between application data and executable expressions.
- Lack of context-aware output encoding.

4. Proof of Concept (PoC)

Authorized lab testing methodology:

1. Open the assigned PortSwigger lab.
2. Identify the reflected input parameter.
3. Determine whether the input is evaluated as an AngularJS expression.
4. Inspect the AngularJS version and expression context.
5. Examine whether the expression evaluation restrictions can be bypassed without using string literals.
6. Test a harmless expression to verify the evaluation behavior.
7. Confirm JavaScript execution using browser developer tools.

Example expression:

```
None
{{1+1}}
```

This is a harmless expression-evaluation test, not a sandbox escape payload.

5. Security Impact

Successful exploitation may result in:

- JavaScript execution in the application's origin.
- Manipulation of application content.
- Unauthorized actions through the victim's session.
- Exposure of information accessible to client-side scripts.
- Potential compromise of application functionality.

6. Recommended Remediation

- Never evaluate untrusted input as an AngularJS expression.
- Avoid relying on AngularJS sandbox restrictions.
- Upgrade legacy AngularJS applications to supported technologies.
- Use safe templating and data-binding mechanisms.
- Apply context-aware output encoding.

- Avoid dynamically compiling untrusted templates.
- Implement a restrictive Content Security Policy as defense in depth.

7. Verification

1. Repeat the original test.
2. Confirm that user input is treated as data.
3. Verify that AngularJS does not evaluate attacker-controlled expressions.
4. Test different expression and encoding variations.
5. Confirm that legitimate application functionality remains intact.

8. References

- PortSwigger: AngularJS Sandbox
- AngularJS Security Guide
- OWASP XSS Prevention Cheat Sheet

9. Disclaimer

For educational purposes and authorized security testing only.